

Computer Hydrological Forecasting

Three independent monthly ARIMA models -- discharge, rainfall, stage -- validated
by stochastic properties, not point comparison

Author: Ugbodaga Benedict Osikpemi (190402003)

Department: Civil and Environmental Engineering

Institution: University of Lagos

Supervisor: Prof. K. O. Aiyesimoju

Submission: February 2026

This document explains the project in three parts: first in very simple terms, then the real technical version, and finally how to actually use the software. It starts with what changed most recently, since that's what makes everything after it correct.

0. What Changed Most Recently (and Why)

Two things changed on 2026-08-12, on the supervisor's direct instruction, and both matter for reading everything that follows correctly.

A data error was found and corrected

Every earlier version of this project used USGS gauge 02361000, believing it was the Conecuh River. It is not: NWIS confirms that gauge is the Choctawhatchee River near Newton, AL, a different river entirely. A search turned up the real Conecuh gauge -- USGS 02371500, Conecuh River at Brantley, AL -- which is also a genuine CAMELS basin, and has a longer, cleaner record than the one mistakenly used before (discharge back to 1937 rather than 1980, still actively updating). All data was re-pulled under the correct gauge, and every reference to the old gauge number in the live code and this report was corrected.

The modelling approach pivoted

- **Monthly, not daily:** Differencing only does useful work -- removing seasonality -- at a monthly timestep, not a daily one, on a river that never really trends day to day.
- **Three variables, not one:** Discharge, rainfall and stage are each modelled independently, to show the same ARIMA method generalises across variable types, not just discharge.
- **Stochastic validation:** A stochastic model's individual forecasts are not meant to match the one thing that actually happened; only its statistical properties -- mean, variability, seasonality, drought duration, peak size -- should match history.

An even earlier version of this project (before either of the above) used a rainfall-runoff model on a Nigerian basin; that was replaced by the ARIMA approach on the same supervisor's instruction, and is not revisited here -- see CLAUDE.md and archive/ for that history if needed.

1. The Simple Version (for anyone)

A river does not change from one month to the next by pure chance. A wet month tends to follow other wet months in the same season; a dry spell tends to persist for a while once it starts. This project builds three small computer models -- one for how much water flows in the river, one for how much rain falls on its catchment, and one for how high the river runs -- and each one learns its own variable's habits from decades of monthly history. None of them needs a weather forecast; each works from its own past alone.

Because rivers, rainfall and river levels are naturally unpredictable (that's what "stochastic" means here), the model doesn't try to name the one exact number that will happen next month. Instead it generates a thousand plausible versions of what could happen, and the honest question asked of it is: do the ordinary and extreme features of those thousand versions -- how wet, how dry, how variable, how seasonal -- look like the real history? That is a much fairer test of a model that admits the future is uncertain than asking it to guess one specific number correctly.

How do we know it works?

For each of the three variables, we held back eleven years of real data (2004-2014) that the model never saw while being built, generated a thousand synthetic versions of what those eleven years could have looked like, and checked seven different properties of the real data -- its average, its variability, how extreme its wettest and driest spells were, and so on -- against the range the thousand synthetic versions produced. Discharge and rainfall passed on all seven; river stage passed on six of seven, and the one it missed (its average level) is reported honestly rather than hidden.

2. The Actual Version (technical)

The project is a modular, open-source Python framework for monthly hydrological forecasting. It fits three independent ARIMA (autoregressive integrated moving average) models, one each for discharge, rainfall and stage, using the Box-Jenkins methodology, and validates each by the statistical properties its stochastic output reproduces relative to the historical record. No variable is used to forecast another, and no exogenous (meteorological forecast) input is used.

2.1 The data

Monthly discharge (m³/s), rainfall (mm/month) and stage (m) for the Conecuh River at Brantley, Alabama (USGS gauge 02371500 -- verified directly against USGS NWIS, see page 2), 1980-2014 (420 months). Discharge and stage come from USGS NWIS; rainfall is the Daymet basin-mean product from the CAMELS archive, chosen because it is the only one of three available rainfall products with zero missing days across the full record. All three series are strictly positive throughout, so each is modelled on the natural-log scale. The record is split into a training period (1980-2003) and an independent validation period (2004-2014).

2.2 The model: three independent ARIMA(p, d, q)

An ARIMA model describes a series using three ingredients: an autoregressive part (the value depends on its own p past values), an integration order d (the series is differenced d times to make it stationary), and a moving-average part (the value depends on the q past random shocks). On the differenced log-series $w(t)$, for each variable independently:

$$w(t) = c + \phi_1 w(t-1) + \dots + \phi_p w(t-p) + a(t) + \theta_1 a(t-1) + \dots + \theta_q a(t-q)$$

where ϕ are the autoregressive coefficients, θ the moving-average coefficients, c a constant, and $a(t)$ a white-noise error term. ARIMAX (which adds an exogenous predictor) was deliberately not used: each variable is forecast from its own past only, by design, so there is no exogenous driver to add.

2.3 How each model was identified (Box-Jenkins)

- **Stationarity:** Augmented Dickey-Fuller and KPSS tests determined the differencing order for each variable independently. For this basin, at the monthly timestep, all three came back $d = 0$.
- **Identification:** The autocorrelation (ACF) and partial autocorrelation (PACF) functions suggested candidate AR and MA orders.
- **Estimation:** Coefficients were estimated by conditional sum of squares (pure AR models by exact least squares), each with a standard error -- answering not just what order was picked, but how precisely each coefficient is known.
- **Selection:** All candidate orders were ranked by the Akaike Information Criterion (AIC).

Discharge : ARIMA(4, 0, 1) AIC=-139.3

Rainfall	: ARIMA(1, 0, 0)	AIC=-258.2
Stage	: ARIMA(4, 0, 0)	AIC=-472.9

2.4 Stochastic ensembles and property-based validation

Rather than one deterministic forecast, each fitted model generates an ensemble of 1,000 synthetic monthly sequences over the validation period. Each sequence, and the actual historical record, is characterised by seven statistics: mean, standard deviation, skewness, month-to-month persistence, seasonal amplitude, longest dry spell, and peak value. A statistic is judged reproduced if the historical value falls within the ensemble's 5th-to-95th-percentile range.

Variable	Properties within 90% envelope
Discharge	7 / 7
Rainfall	7 / 7
Stage	6 / 7

Stage's one miss (its mean) is traced to residual seasonal structure the non-seasonal ARIMA specification does not fully capture -- visible independently as a failed Ljung-Box residual test for discharge and stage ($p < 0.05$, some autocorrelation remains) but not for rainfall ($p = 0.960$). This is reported as an acknowledged limitation, not concealed: a validation procedure that always passes would not be doing useful work.

2.5 Code structure

```
src/preprocess.py - monthly loaders: discharge, rainfall, stage
src/model.py      - ARIMA + ADF/KPSS/ACF/PACF/Ljung-Box + std. errors
src/calibrate.py  - stationarity tests + AIC order selection
src/simulate.py   - stochastic synthetic-ensemble generation
src/validation.py - property-based validation vs. historical record
src/plots.py      - the six figures (3-variable, monthly, stochastic)
run_pipeline.py   - runs everything end to end, all 3 variables
write_full_report.py + report_*.py - builds the full Ch1-5 report
app.py + pages/   - the Streamlit web app
```

The entire time-series toolkit -- ARIMA, stationarity tests, ACF/PACF, Ljung-Box, standard errors, stochastic simulation, property-based validation -- is implemented directly from first principles on NumPy and SciPy, so the framework is fully self-contained and reproducible (no external time-series library is required).

3. How to Use the Software

3.0 The easy version

Open the app, pick a variable (discharge, rainfall or stage), pick a starting month and how many months ahead to look, and press Run Forecast. You'll see a band of plausible futures, not one line -- and that band will look slightly different every time you press the button, because the model is honestly stochastic. Here is all you do:

- 1. Open the app - a page opens in your web browser.
- 2. On the left, pick the variable and the forecast origin month.
- 3. Choose the horizon (months ahead) and how many synthetic replicates to generate.
- 4. Press the "Run Forecast" button.
- 5. Read the answer: a band of plausible outcomes as cards, a table, a chart, and the property-based validation results.

3.1 The web app, step by step (the main way to use it)

Open the app and you get a browser page with two pages in the left sidebar: "Forecast Tool" and "Documentation". On the Forecast Tool page:

- 1. Choose the Variable (discharge, rainfall or stage).
- 2. Choose the Forecast origin (month) and Horizon (months ahead).
- 3. Click "Run Forecast".
- 4. Read the results: per-month summary cards showing the median and 90% band, a forecast table, a chart of recent history plus the synthetic band, and the property-based validation table showing how many statistics the model reproduces.

3.2 The full run (get every chart and number at once)

Run the whole study in one command. It loads all three monthly series, runs the stationarity tests, identifies and estimates all three ARIMA models with standard errors, generates the stochastic ensembles, runs property-based validation, and saves all six figures plus a results file (data/results.json).

```
python run_pipeline.py      # all 3 models + ensembles + figures
python write_full_report.py # rebuild the full Ch1-5 report .docx
python make_overview_pdf.py # rebuild this overview PDF
streamlit run app.py        # launch the interactive web app
```